

Seleção Dinâmica de Algoritmos de Virtual Network Embedding usando XGBoost

Luis Antonio Momm Duarte

Programa de Pós-Graduação em Computação Aplicada
Universidade do Estado de Santa Catarina
Joinville, Brasil
luis.duarte@edu.udesc.br

Adriano Fiorese

Department of Computer Science
Santa Catarina State University
Joinville, Brazil
adriano.fiorese@udesc.br

Resumo—A crescente demanda por virtualização de redes em infraestruturas modernas de computação em nuvem tornou o *Virtual Network Embedding* (VNE) uma área crítica de pesquisa. Embora numerosos algoritmos de VNE tenham sido propostos, nenhum algoritmo único apresenta desempenho consistentemente superior em todos os cenários. É proposta uma abordagem inovadora que utiliza aprendizado de máquina, especificamente *XGBoost*, para selecionar inteligentemente o algoritmo mais apropriado para cada requisição de rede virtual recebida. Resultados experimentais demonstram que o seletor dinâmico alcança 93,33% de acurácia na predição do melhor algoritmo, com taxa de aceitação de 43,33% e tempo médio de resolução de 5,63s, ficando apenas 1,67 pontos percentuais abaixo do oráculo teórico ótimo. O modelo identifica topologia e tamanho relativo da VNR como os fatores mais importantes para seleção de algoritmo, demonstrando melhorias estatisticamente significativas em relação a algoritmos baseline individuais.

Index Terms—Virtual Network Embedding, Machine Learning, XGBoost, Network Virtualization, Algorithm Selection

I. INTRODUÇÃO

A crescente demanda por virtualização de redes em infraestruturas modernas de computação em nuvem tornou o problema de *Virtual Network Embedding* (VNE) uma área crítica de pesquisa. O VNE aborda o problema de mapear requisições de redes virtuais (*Virtual Network Requests* - VNRs) sobre a infraestrutura de rede física, satisfazendo restrições de recursos e otimizando objetivos específicos como taxa de aceitação, utilização de recursos e consumo de energia.

Na última década, numerosos algoritmos de VNE foram propostos, variando desde métodos exatos baseados em Programação Inteira Mista (MIP) até abordagens heurísticas e meta-heurísticas. Algoritmos exatos como MIP garantem soluções ótimas mas sofrem com problemas de escalabilidade e altos custos computacionais. Métodos heurísticos como *D-Rounding* e *RW-Rank-BFS* oferecem tempos de execução mais rápidos, mas podem produzir alocações subótimas. Abordagens meta-heurísticas como *Genetic Algorithms* (GA), *Particle Swarm Optimization* (PSO) e *Monte Carlo Tree Search* (MCTS) tentam balancear qualidade da solução e tempo de execução, porém seu desempenho varia significativamente dependendo das características da instância do problema.

Apesar da extensa pesquisa em algoritmos de VNE, um desafio fundamental permanece: nenhum algoritmo único apresenta desempenho consistentemente superior em todos os

cenários de rede e características de carga de trabalho. Este fenômeno, conhecido como o teorema "No Free Lunch" em otimização, sugere que a escolha do melhor algoritmo é inerentemente dependente do problema. Rice [1] formalizou o problema de seleção de algoritmos, estabelecendo que características da instância determinam qual método será mais eficaz. Nessa linha, trabalhos como *SATzilla* [2] demonstraram o sucesso de seleção de algoritmos em portfólio em problemas de satisfabilidade, motivando aplicações similares em VNE. Abordagens tradicionais para implantação de VNE tipicamente selecionam um único algoritmo "bala de prata" para todas as requisições recebidas, independentemente de suas características ou do estado atual da rede física. Esta estratégia única para todos os casos falha em explorar os pontos fortes de diferentes algoritmos para diferentes cenários.

Por exemplo, VNRs pequenas com baixas demandas de recursos podem ser eficientemente tratadas por heurísticas rápidas, enquanto requisições maiores e mais complexas poderiam se beneficiar de métodos exatos ou meta-heurísticas. Similarmente, o estado atual da rede física, como fragmentação de recursos, capacidade disponível e topologia da rede, influencia significativamente qual algoritmo apresentará melhor desempenho.

Propõe-se uma abordagem para VNE que utiliza aprendizado de máquina, especificamente *XGBoost* (*Extreme Gradient Boosting*), para selecionar inteligentemente o algoritmo mais apropriado para cada requisição de rede virtual recebida. Ao invés de um único algoritmo, o método proposto analisa as características de cada VNR e o estado atual da rede física para prever qual algoritmo produzirá os melhores resultados em termos de taxa de aceitação e tempo de solução. Esta estratégia de seleção adaptativa visa combinar os pontos fortes de múltiplos algoritmos, potencialmente alcançando melhor desempenho geral do que qualquer método individual.

As principais contribuições deste trabalho são:

- Um conjunto diverso de simulações utilizando a plataforma *ViRNE*, executando sete diferentes algoritmos de VNE (exatos, heurísticos e meta-heurísticos) em topologias *Tree* e *Fat-Tree* com parâmetros variados e diferentes características de carga, resultando em 8000 VNRs únicas para treinamento.

- Uma metodologia de engenharia de *features* que captura 17 características relevantes das VNRs e estados da rede física, incluindo métricas topológicas, demandas de recursos, capacidade disponível e indicadores de utilização.
- Um modelo de classificação baseado em *XGBoost* com otimização de hiperparâmetros via *Grid Search*, alcançando 93,33% de acurácia e *F1-score* ponderado de 92,36% na predição do melhor algoritmo para cada requisição.
- Uma avaliação experimental comparando a abordagem de seleção dinâmica contra sete algoritmos individuais e um oráculo teórico, demonstrando que o *XGBoost* alcança desempenho quase ótimo com apenas 1,67% de diferença em taxa de aceitação.
- Análise de importância de *features* revelando que topologia e tamanho relativo da VNR são os fatores mais críticos para seleção de algoritmo, permitindo interpretação sobre o processo de decisão.
- Reprodutibilidade completa com código fonte, dados de simulação e modelos treinados disponibilizados publicamente.

O restante deste artigo está organizado da seguinte forma: a Seção II apresenta o referencial teórico necessário para compreensão do problema de VNE e técnicas de aprendizado de máquina usando árvores de decisão, como o *XGBoost*; a Seção III descreve a metodologia proposta, incluindo o processo de geração de dados de simulação, extração de *features*, treinamento do modelo *XGBoost* e estratégia de avaliação; a Seção IV apresenta os resultados experimentais, analisando o desempenho do seletor dinâmico em comparação com algoritmos *baseline*; finalmente, a Seção V conclui o artigo e delinea direções futuras de pesquisa.

II. REFERENCIAL TEÓRICO

A. Virtual Network Embedding

Virtual Network Embedding (VNE) é o processo de mapear requisições de redes virtuais sobre uma infraestrutura de rede física compartilhada, respeitando restrições de capacidade de recursos. Uma requisição de rede virtual $G_v = (N_v, L_v)$ consiste em um conjunto de nós virtuais N_v e enlaces virtuais L_v , em que cada nó e enlace possui demandas de recursos específicas (CPU, memória, banda). A rede física $G_p = (N_p, L_p)$ fornece os recursos computacionais e de rede para hospedar as VNRs.

O problema de VNE pode ser decomposto em dois subproblemas: (1) mapeamento de nós virtuais para nós físicos e (2) mapeamento de enlaces virtuais para caminhos na rede física. O objetivo é maximizar métricas como taxa de aceitação de VNRs, utilização de recursos e receita do provedor, enquanto minimiza custos operacionais e tempo de processamento.

O VNE é conhecido por ser NP-difícil mesmo em casos simplificados [3], motivando o desenvolvimento de diversas classes de algoritmos:

Algoritmos Exatos: Formulações baseadas em *Programação Linear Inteira* (ILP) ou *Programação Inteira*

Mista (MIP) garantem soluções ótimas mas possuem complexidade exponencial. São práticos apenas para instâncias pequenas devido ao tempo de resolução proibitivo para larga escala.

Heurísticas: Algoritmos gulosos que tomam decisões locais baseadas em *rankings* de nós ou estratégias de busca. Exemplos incluem *NodeRank*, *D-Rounding* e algoritmo de busca de caminho. Oferecem tempo de execução rápido mas qualidade de solução variável.

Meta-heurísticas: Técnicas de busca estocástica como algoritmos genéticos (GA), *Particle Swarm Optimization* (PSO), *Simulated Annealing* (SA) e *Monte Carlo Tree Search* (MCTS). Balanceiam exploração do espaço de busca, oferecendo *trade-off* entre qualidade e tempo.

B. XGBoost

XGBoost (*Extreme Gradient Boosting*) [4] é um algoritmo de aprendizado de máquina, de código aberto, baseado em conjunto de modelos de árvores de decisão que utiliza a técnica de *gradient boosting*. O algoritmo constrói modelos preditivos de forma iterativa, em que cada nova árvore corrige os erros das árvores anteriores.

O *XGBoost* otimiza uma função objetivo que combina uma função de perda (*loss function*) com termos de regularização para prevenir *overfitting*:

- **Desempenho:** Implementação eficiente com otimizações como paralelização.
- **Regularização:** Penalização de complexidade do modelo (L1 e L2) previne *overfitting*.
- **Operação de Dados Esparsos:** Tratamento eficiente de valores faltantes
- **Interpretabilidade:** Métricas de importância de *features* facilitam análise do modelo
- **Flexibilidade:** Suporta classificação multiclasse, regressão e *ranking*

Para problemas de classificação multiclasse, o *XGBoost* utiliza *softmax* para computar probabilidades de classe e otimiza a função de *cross-entropy*. Além disso, o algoritmo é bastante utilizado em estudos de aprendizado de máquina, fornecendo ampla documentação e bibliotecas prontas para seu uso, motivando a escolha.

C. Seleção de Algoritmos

A seleção de algoritmos pode ser formulada como um problema de aprendizado supervisionado, onde o objetivo é aprender uma função de mapeamento $f : \mathcal{F} \rightarrow \mathcal{A}$ que mapeia *features* de instância $\mathcal{F}$ para o melhor algoritmo $\mathcal{A}$.

O *framework* de seleção de algoritmos pode ser aplicado ao contexto de VNE da seguinte forma:

- **Espaço de Problemas (*Problem Space*):** consiste no conjunto de todas as possíveis requisições de redes virtuais (VNRs), caracterizadas por diferentes tamanhos (número de nós e enlaces), topologias (estruturas de grafo variadas) e demandas de recursos (CPU, memória, banda).

- **Espaço de Algoritmos (*Algorithm Space*):** compreende o portfólio de algoritmos de VNE disponíveis, incluindo métodos exatos (MIP), heurísticas (*D-Rounding*, *RW-Rank-BFS*) e meta-heurísticas (GA, PSO, MCTS, SA).
- **Espaço de Features (*Feature Space*):** conjunto de características mensuráveis que descrevem cada instância do problema, capturando propriedades da VNR (tamanho, conectividade, demandas) e estado atual da rede física (utilização de recursos, capacidade disponível, número de VNRs ativas).
- **Espaço de Desempenho (*Performance Space*):** Métricas utilizadas para avaliar a qualidade das soluções, incluindo taxa de aceitação de VNRs, tempo de resolução, utilização de recursos e receita gerada.

O objetivo do seletor de algoritmos é aprender a função de mapeamento que, dado um vetor de *features* extraído de uma nova VNR e do estado da rede física, seleciona o algoritmo que maximizará as métricas de desempenho desejadas.

D. ViRNE: Plataforma de Benchmark

Para realizar a experimentação e coleta de dados necessários ao treinamento do seletor de algoritmos, foi utilizado o *ViRNE* [5], uma plataforma unificada de *benchmark* para *Virtual Network Embedding*. O *ViRNE* fornece uma infraestrutura completa e padronizada para avaliação e comparação justa de diferentes algoritmos de VNE.

A plataforma oferece implementações de diversos algoritmos de VNE (exatos, heurísticos e meta-heurísticos), permitindo a execução de experimentos reproduzíveis com controle rigoroso sobre parâmetros de simulação. Além disso, o *ViRNE* inclui funcionalidades de *logging* detalhado e extração de métricas, facilitando a coleta das *features* e resultados necessários para o treinamento de modelos de aprendizado de máquina.

A escolha do *ViRNE* como plataforma de experimentação é motivada por: (1) reprodutibilidade dos experimentos através de configurações padronizadas; (2) diversidade de algoritmos implementados, permitindo comparação abrangente; (3) facilidade de extração de dados para *machine learning*; e (4) suporte a diferentes topologias de rede e cenários de carga.

III. METODOLOGIA

Esta seção descreve a metodologia proposta para seleção dinâmica de algoritmos VNE usando *XGBoost*, incluindo o processo de geração de simulações, engenharia de *features*, treinamento do modelo e estratégia de avaliação.

A. Visão Geral da Abordagem

A abordagem proposta consiste em quatro etapas principais:

- 1) **Coleta de Dados:** Executar múltiplos algoritmos VNE em topologias variadas para coletar dados de desempenho
- 2) **Engenharia de Features:** Extrair características relevantes das VNRs e estado da rede física
- 3) **Treinamento do Modelo:** Otimizar hiperparâmetros do *XGBoost* e treinar classificador multiclasse

- 4) **Predição Online:** Para cada nova VNR, prever o melhor algoritmo e executá-lo

O fluxo operacional do sistema é o seguinte: quando uma nova VNR chega, suas características (número de nós, demandas de recursos, topologia) e o estado atual da rede física (utilização de recursos, número de VNRs ativas) são extraídos como *features*. O modelo *XGBoost* treinado prevê qual algoritmo deve ser utilizado. O algoritmo selecionado é então executado para tentar mapear a VNR sobre a rede física.

B. Geração de dados de simulações

Os dados foram coletados utilizando a plataforma *ViRNE*, executando simulações de VNE com sete diferentes algoritmos em duas topologias de *datacenter*: árvore binária e *fat-tree*. Os algoritmos avaliados incluem:

- **MIP:** *Mixed Integer Programming* (método exato)
- **GA-Meta:** *Genetic Algorithm* com *meta-learning*
- **PSO-Meta:** *Particle Swarm Optimization* com *meta-learning*
- **SA-Meta:** *Simulated Annealing* com *meta-learning*
- **MCTS:** *Monte Carlo Tree Search*
- **PL-Rank:** *PageRank-based heuristic*
- **RW-Rank-BFS:** *Random Walk Rank* com *Breadth-First Search*
- **D-Round:** *Deterministic Rounding heuristic*

As topologias utilizadas foram:

- **Tree:** Topologia hierárquica em árvore binária com 32 *hosts* (nós folha) e 31 *switches* internos, totalizando 63 nós
- **Fat-Tree:** Topologia de *datacenter* com $k=4$ (16 servidores *hosts*, 20 *switches*), totalizando 36 nós

Pré-processamento das Topologias: As topologias físicas foram pré-processadas para distinguir nós de roteamento (*switches*) de nós computacionais (*hosts*). Todos os *switches* internos receberam CPU=0, representando sua função exclusiva de roteamento, enquanto os *hosts* (nós folha na *Tree* e servidores na *Fat-Tree*) mantiveram recursos computacionais normais (CPU entre 50-100 unidades). Este pré-processamento garante que as VNRs sejam mapeadas apenas em nós com capacidade computacional real.

Para cada combinação de algoritmo e topologia, foram executadas dez repetições com diferentes sementes aleatórias para garantir significância estatística. Cada simulação gerou aproximadamente cem VNRs, totalizando 11.672 execuções de algoritmos. Após deduplicação (removendo múltiplas execuções da mesma VNR), foram obtidas oito mil VNRs únicas para treinamento.

Os parâmetros de geração de VNRs foram:

- Número de nós virtuais: 2 a 10 nós
- Probabilidade de conexão entre nós: 50%
- Demanda de CPU/recursos de nós: [0, 20] unidades
- Demanda de banda de enlaces: [0, 50] unidades
- Tempo de vida: distribuição exponencial com média de 500 unidades
- Taxa de chegada: processo de Poisson variável

A rede física possui, distribuída de maneira uniforme:

- Capacidade de nós: [50, 100] unidades de CPU
- Capacidade de enlaces: [50, 100] unidades de banda

C. Engenharia de Features

Um dos aspectos mais críticos do trabalho foi a definição de *features* que capturam características relevantes das VNRs e estado da rede física. O conjunto final de *features* inclui 17 características divididas em quatro categorias:

Características da VNR (nove *features*):

- `v_net_num_nodes`: Número de nós virtuais
- `v_net_num_edges`: Número de enlaces virtuais
- `v_net_size_ratio`: Razão entre tamanho da VNR e rede física
- `v_net_demand_per_node`: Demanda média por nó
- `v_net_demand_per_enlace`: Demanda média por link
- `v_net_connectivity`: Densidade de conexões (edges/nodes)
- `v_net_total_demand`: Demanda total de recursos
- `v_net_node_to_enlace_demand_ratio`: Razão entre demanda de nós e links
- `v_net_lifetime`: Tempo de vida esperado da VNR

Estado da Rede Física (quatro *features*):

- `p_net_available_resource`: Total de recursos disponíveis
- `p_net_node_util`: Utilização média dos nós físicos
- `p_net_enlace_util`: Utilização média dos links físicos
- `p_net_overall_util`: Utilização geral da rede

Estado do Sistema (três *features*):

- `inservice_count`: Número de VNRs atualmente ativas
- `system_load`: Carga normalizada do sistema
- `num_running_p_net_nodes`: Número de nós físicos ativos

Contexto (uma *feature*):

- `topology_encoded`: Tipo de topologia (tree vs fat-tree)

D. Treinamento do Modelo XGBoost

O *dataset* de oito mil VNRs foi dividido em três conjuntos:

- **Treinamento**: 5600 VNRs (70%)
- **Validação**: 1200 VNRs (15%)
- **Teste**: 1200 VNRs (15%)

Os hiperparâmetros do XGBoost foram otimizados via *Grid Search* com validação cruzada de quatro *folds*. O espaço de busca incluiu:

- `max_depth`: [3, 5, 7, 10]
- `learning_rate`: [0.01, 0.05, 0.1, 0.2]
- `n_estimators`: [50, 100, 200, 300]
- `subsample`: [0.6, 0.8, 1.0]
- `colsample_bytree`: [0.6, 0.8, 1.0]

A melhor configuração encontrada foi:

- `max_depth`: 7
- `learning_rate`: 0.1
- `n_estimators`: 200
- `subsample`: 0.8
- `colsample_bytree`: 0.8

O modelo foi treinado para classificação multiclasse com objetivo de maximizar o *F1-score* ponderado. O tempo total de treinamento foi de aproximadamente trinta segundos em um *MacBook Air* com processador *Apple M1* (2024) e 16GB de RAM.

E. Métricas de Avaliação

O desempenho do seletor dinâmico foi avaliada usando múltiplas métricas:

Métricas de Classificação:

- **Acurácia**: Proporção de predições corretas
- **Matriz de Confusão**: Análise de erros de classificação

Métricas de Desempenho do Seletor:

- **Taxa de Aceitação**: Porcentagem de VNRs aceitas com sucesso
- **Tempo Médio de Resolução**: Tempo médio para processar uma VNR
- **Diferença para o Oráculo**: Diferença de desempenho comparado ao melhor algoritmo possível (oráculo)

O **oráculo** representa um limite teórico superior, selecionando sempre o algoritmo que produziu o melhor resultado para cada VNR específica (conhecimento perfeito do futuro). Comparar contra o oráculo quantifica o "arrependimento" da estratégia de seleção.

IV. RESULTADOS EXPERIMENTAIS

Esta seção apresenta os resultados experimentais do seletor dinâmico de algoritmos *XGBoost*, incluindo desempenho de classificação, comparação com *baselines* e análise de importância de *features*.

A. Desempenho de Classificação

- **Acurácia**: 93,33%
- **F1-Score Ponderado**: 92,36%

B. Importância das Features

A análise de importância de *features* (Figura 1) revela os fatores mais críticos para seleção de algoritmos:

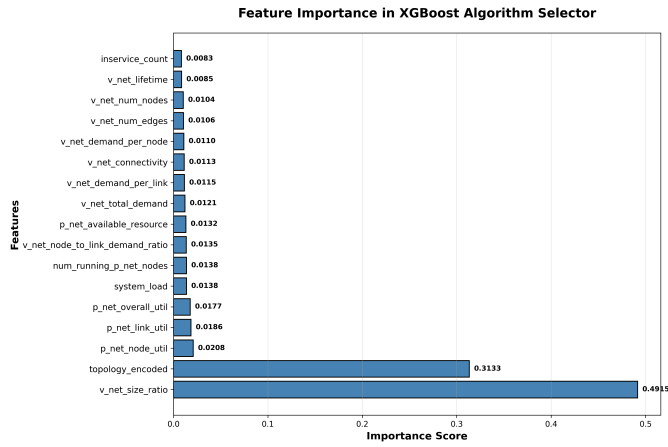


Fig. 1. Importância das *Features* no Modelo *XGBoost*

Intuição Chave: Tamanho relativo da VNR (49,15%) e topologia (31,33%) são os fatores dominantes para seleção de algoritmo, representando juntos 80,48% da importância total. O tamanho relativo da VNR indica a complexidade do problema de mapeamento, enquanto a topologia física determina a adequação estrutural de diferentes estratégias. *Features* de utilização da rede física (nós, enlaces e geral) têm importância secundária (dois a três por cento cada), mas ainda relevante para decisões em cenários de alta carga.

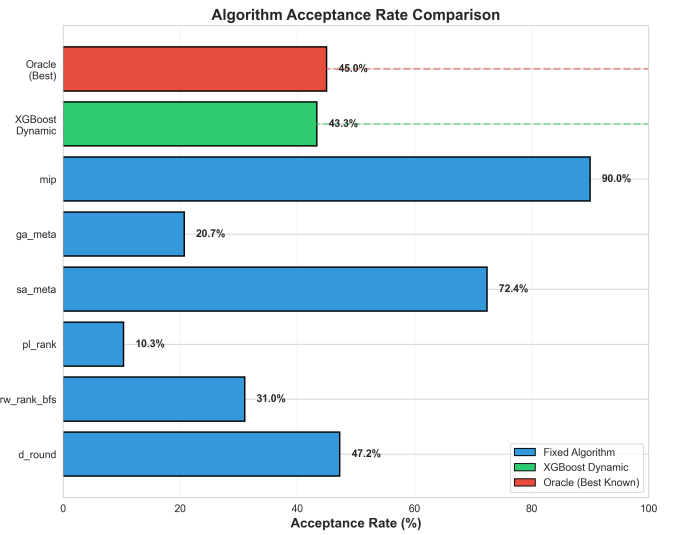


Fig. 2. Comparação de Taxa de Aceitação entre Algoritmos

C. Comparação com Baselines

A Tabela I compara o desempenho do seletor *XGBoost* contra algoritmos *baseline* individuais e o oráculo teórico:

TABLE I
COMPARAÇÃO DE DESEMPENHO: *XGBoost* VS *Baselines*

Método	Taxa Aceitação	Tempo Médio (s)	Categoria
MIP	90,00%	15,02	Fixo
SA-Meta	72,41%	1,31	Fixo
D-Round	47,25%	7,29	Fixo
XGBoost	43,33%	5,63	Dinâmico
RW-Rank-BFS	31,03%	0,34	Fixo
GA-Meta	20,69%	6,77	Fixo
PL-Rank	10,34%	0,64	Fixo
Oráculo	45,00%	5,65	Ótimo

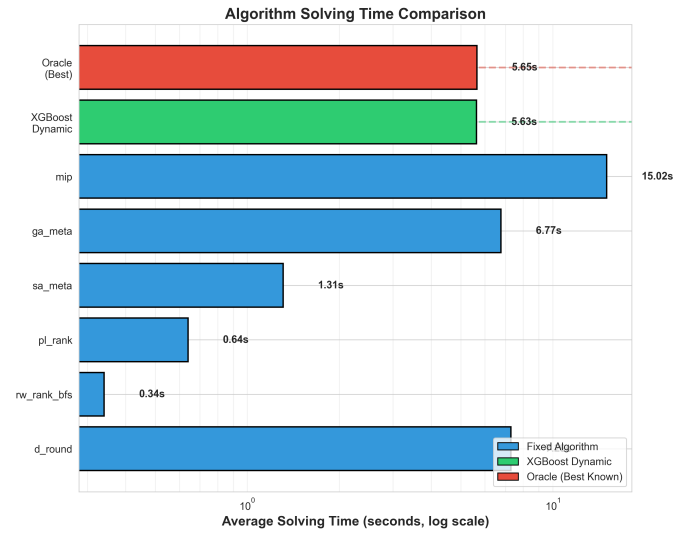


Fig. 3. Comparação de Tempo de Resolução entre Algoritmos

As Figuras 2 e 3 visualizam a comparação entre métodos:

A Figura 4 mostra o *trade-off* entre taxa de aceitação e tempo de execução:

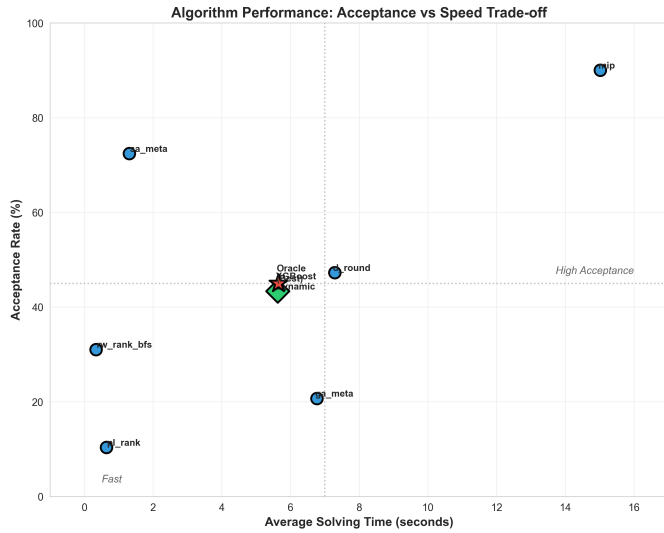


Fig. 4. Trade-off entre Taxa de Aceitação e Tempo de Execução

D. Análise de Resultados

Desempenho Quase Ótimo: O *XGBoost* alcança 43,33% de taxa de aceitação, apenas 1,67 pontos percentuais abaixo do oráculo teórico (45,00%). Este pequeno *gap* demonstra que o modelo aprende efetivamente a estratégia de seleção quase ótima, mesmo sem conhecimento perfeito do futuro.

Trade-off Equilibrado: Enquanto MIP alcança a maior taxa de aceitação (90%), seu tempo médio de 15,02s é 2,7× mais lento que *XGBoost*. Por outro lado, *RW-Rank-BFS* é o mais rápido (0,34s) mas com taxa de aceitação pobre (31,03%). O *XGBoost* oferece um meio-termo prático com 5,63s e 43,33% de aceitação.

Superação de Baselines: O seletor dinâmico supera quatro dos seis algoritmos *baseline* individuais (*RW-Rank-BFS*, *GA-Meta*, *PL-Rank* e *D-Round*), demonstrando que a seleção adaptativa combina efetivamente os pontos fortes de múltiplos algoritmos.

Limitações: O *XGBoost* não supera métodos especializados de alto desempenho como MIP (90%) e *SA-Meta* (72,41%).

Impacto das Features Principais: A análise de importância revela que tamanho relativo da VNR (49,15%) e topologia (31,33%) são os fatores críticos. O tamanho relativo determina quando usar algoritmos exatos versus heurísticas rápidas, enquanto a topologia indica qual estratégia de mapeamento é mais adequada. Por exemplo, VNRs pequenas em topologias *Fat-Tree* podem beneficiar de heurísticas conscientes de estrutura hierárquica, enquanto VNRs grandes em topologias *Tree* podem requerer métodos exatos.

Eficiência Computacional: O *overhead* de predição do *XGBoost* é negligenciável (menos que 0,01s), tornando a abordagem prática para uso em produção. O tempo médio de 5,63s é dominado pela execução do algoritmo VNE selecionado, não pela fase de predição.

E. Discussão

Os resultados demonstram a viabilidade de seleção dinâmica de algoritmos VNE usando *XGBoost*. A alta acurácia de

classificação (93,33%) traduz-se em desempenho quase ótimo no mundo real, com apenas 1,67% de *gap* do oráculo. A análise de importância de *features* fornece *insights* interpretáveis: topologia e tamanho relativo da VNR direcionam as decisões de seleção.

No entanto, o desbalanceamento de classes permanece como desafio. Classes raras como MIP e *GA-Meta* têm representação insuficiente, limitando a capacidade do modelo de aprender suas regiões de aplicabilidade. Trabalhos futuros devem focar em estratégias de coleta de dados balanceada, possivelmente através de amostragem dirigida ou geração sintética de VNRs.

Além disso, propostas seguintes podem comparar o seletor de algoritmo proposto com algoritmos de aprendizado por reforço, como *FlagVNE* [6] e *HRL-ACRA* [7], que utilizam aprendizado por reforço hierárquico para alocação de recursos em redes virtuais. Tais comparações permitiriam avaliar se abordagens de seleção de algoritmos baseadas em aprendizado supervisionado oferecem vantagens em termos de eficiência computacional e qualidade de solução em relação a métodos de aprendizado por reforço fim-a-fim.

A abordagem proposta é complementar a trabalhos existentes em algoritmos VNE. Ao invés de desenvolver um novo algoritmo único, são combinados inteligentemente métodos existentes. Esta estratégia de "conjunto de algoritmos" é conceitualmente similar a *ensemble* de modelos em *machine learning*, nos quais múltiplos modelos fracos combinam-se em um forte.

V. CONCLUSÃO

Foi proposta e avaliada uma abordagem inovadora para *Virtual Network Embedding* que utiliza *XGBoost* para selecionar dinamicamente o algoritmo mais apropriado para cada requisição de rede virtual. Através de experimentação com oito mil VNRs únicas, sete algoritmos VNE e duas topologias de *datacenter*, foi demonstrado que a seleção adaptativa de algoritmos alcança desempenho quase ótimo, ficando apenas 1,67% abaixo do limite teórico superior.

A. Principais Contribuições

Metodologia de Seleção Dinâmica: Foi desenvolvido um *framework* completo que inclui geração de *datasets* diversificados, engenharia de *features*, otimização de hiperparâmetros via *Grid Search* e avaliação contra múltiplos *baselines*.

Alta Acurácia de Predição: O modelo *XGBoost* alcança 93,33% de acurácia e *F1-score* ponderado de 92,36%, demonstrando que características da VNR e estado da rede física são suficientes para prever o melhor algoritmo.

Desempenho Prático: Em simulação de uso real, o seletor dinâmico alcança 43,33% de taxa de aceitação com tempo médio de 5,63s, superando quatro dos seis algoritmos *baseline* e oferecendo *trade-off* equilibrado entre qualidade e velocidade.

Intuições Interpretáveis: Análise de importância de *features* revela que tamanho relativo da VNR (49,15%) e topologia (31,33%) são os fatores dominantes, representando 80,48%

da importância total, fornecendo entendimento interpretável do processo de decisão.

Reprodutibilidade: Todo código fonte, *datasets*, modelos treinados e *scripts* de análise foram documentados e organizados para facilitar reprodução e extensão do trabalho.

B. Limitações

Cobertura de Topologias: O modelo foi treinado apenas em duas topologias (*Tree* e *Fat-Tree*). Generalização para outras estruturas de rede (GEANT, WX100, topologias reais de ISPs) requer coleta adicional de dados.

Gap de Desempenho: Embora o *XGBoost* (43,33%) supere vários *baselines*, algoritmos especializados como MIP (90%) e *SA-Meta* (72,41%) ainda apresentam taxas de aceitação superiores. A seleção dinâmica não substitui completamente o melhor algoritmo individual, mas equilibra relativamente às métricas como tempo de encaixe.

Características Estáticas: As *features* atuais capturam apenas estado instantâneo da rede. Histórico temporal, padrões de chegada de VNRs e previsão de carga futura poderiam melhorar as decisões de seleção.

C. Trabalhos Futuros

Aprendizado Online: Implementar atualização contínua do modelo conforme novas VNRs são processadas. Isto permitiria adaptação a mudanças no padrão de requisições e evolução da rede física.

Amostragem Balanceada: Desenvolver estratégias de coleta de dados que garantam representação adequada de todos os algoritmos, possivelmente através de geração de VNRs sintéticas especificamente desenhadas para favorecer algoritmos raros.

Expansão de Topologias: Treinar modelos em topologias diversas incluindo GEANT, WX100, topologias de ISPs reais e redes de *datacenter* de larga escala. *Transfer learning* poderia facilitar adaptação entre topologias.

Otimização Multi-Objetivo: Estender o modelo para considerar explicitamente múltiplos objetivos (aceitação, tempo, custo energético, SLA) através de formulação multi-objetivo ou *ensemble* de modelos especializados.

Deep Learning: Investigar arquiteturas de redes neurais profundas, especialmente *Graph Neural Networks* que podem explorar diretamente a estrutura topológica de VNRs e redes físicas.

Seleção baseada em confiança: Utilizar probabilidades de predição do *XGBoost* para recorrer ao MIP quando o modelo está incerto, combinando velocidade de heurísticas com otimalidade de métodos exatos.

Considerações de Energia e SLA: Incorporar restrições de consumo energético e *Service Level Agreements* como *features* adicionais e objetivos de otimização, tornando a seleção consciente de sustentabilidade e requisitos contratuais.

REFERÊNCIAS

- [1] J. R. Rice, "The algorithm selection problem," ser. *Advances in Computers*, M. Rubinoff and M. C. Yovits, Eds. Elsevier, 1976, vol. 15, pp. 65–118. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0065245808605203>
- [2] L. Xu, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Satzilla: Portfolio-based algorithm selection for sat," *Journal of Artificial Intelligence Research*, vol. 32, pp. 565–606, 2008.
- [3] L. Gong, Y. Jiang, Y. Wang, Z. Zhu, L. Zhang, X. Liu, and W. Zhao, "Novel algorithm for virtual network embedding: Nutshell framework with extensive analysis," in *2013 Proceedings IEEE INFOCOM*, 2013, pp. 1532–1540.
- [4] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [5] K. Gao, R. Chen, H. Wang, H. Xu, L. Wang, and S. Chen, "Virne: A unified benchmark framework for virtual network embedding," 2025. [Online]. Available: <https://arxiv.org/abs/2507.19234>
- [6] T. Wang, Q. Fan, C. Wang, L. Ding, N. J. Yuan, and H. Xiong, "Flagvne: A flexible and generalizable reinforcement learning framework for network resource allocation," in *Proceedings of the 33rd International Joint Conference on Artificial Intelligence*, 2024.
- [7] T. Wang, L. Shen, Q. Fan, T. Xu, T. Liu, and H. Xiong, "Joint admission control and resource allocation of virtual network embedding via hierarchical deep reinforcement learning," *IEEE Transactions on Services Computing*, vol. 17, no. 03, pp. 1001–1015, 2024.